

https://docs.google.com/presentation/d/1hY1lcNvcWUMfN3an7kn0-hnaFx-dfLqyslw2nz7BEbQ/edit#slide=id.g51da6455d5_0_0

Intro To Rust

Course Leads:

Sara Magaziotis-Ginori

Olivia Li

Isaac Oh

Aman Shah

Elson Liu



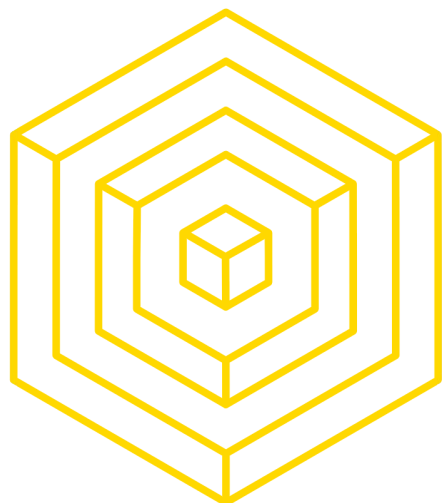
BLOCKCHAIN
AT BERKELEY

Fujia Wang



LECTURE OUTLINE

- 1 Why Rust?
- 2 Memory Management
- 3 Memory Management in Rust
- 4 Other Rust Syntax



1 Why Rust?



Why Rust?

Creation

4

- Rust was started in 2006 by Graydon Hoare, a developer at Mozilla
- He saw an issue with his elevator and realized that problems like this usually come from memory management issues in languages like C and C++
- In 2009 Mozilla started assisting with the project
- In May 2015 Rust 1.0 is released



Source: <https://github.blog/2023-08-30-why-rust-is-the-most-admired-language-among-developers/>

AUTHOR: Jaylem Brar



Why Rust?

Advantages

- Rust is considered a fast language and can be used in projects where performance is key which languages like C and C++ are typically used for
- No garbage collection makes memory management easier and prevents the need to manually free memory
- The improvement in memory management is one of the most regarded features
- This is solved with the ownership and borrowing system to be discussed later today

Source: <https://github.blog/2023-08-30-why-rust-is-the-most-admired-language-among-developers/>

AUTHOR: Jaylem Brar



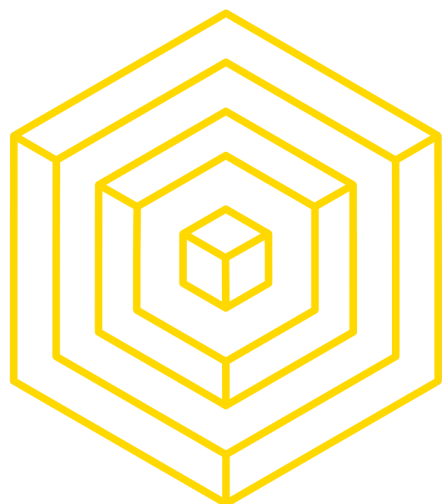
Why Rust?

Advantages

- Good well developed built in package manager
- Abstractions don't slow down performance
- Solid built in support for concurrency
- More type inference than languages like C++ and C

Source: <https://github.blog/2023-08-30-why-rust-is-the-most-admired-language-among-developers/>

AUTHOR: Jaylem Brar



Why Rust?

 Factors	 Rust	 Go
Performance	Rust is better than Go	Go is not faster
Concurrency	Rust lacks concurrency	Build-in Concurrency
Memory Management	The program cannot compile if not memory safe	Go handles memory automatically
Development Speed	Development needs more time	Golang web development is speedy
Compilation Speed	Low	Faster
Libraries	Good Libraries	Good Libraries
Complexity	Rust is complex to understand	Go is a simple language
Functionality	Many features and functionalities	Fewer functionalities

Source: <https://spectralops.io/blog/rust-vs-go-why-not-use-both/>

AUTHOR: Jaylem Brar



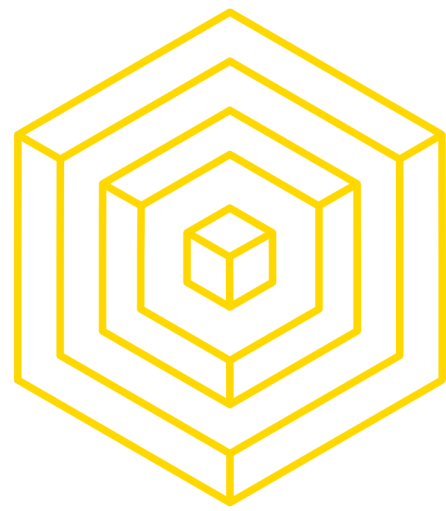
Why Rust?

Feature	C	Rust
Hardware access	Y	Y
Pointers	Y	Y
Link with C binary	Y	Y
Garbage collection	N	N
Zero Cost Abstraction	Y	Y
LLVM Backend	Y	Y
Standard Library	Y	Y
Memory safety	N	Y

Feature	C	Rust
Type inference	N	Y
Parametric types	N	Y
Mono-morphisation	N	Y
Enumerated types	N	Y
Pattern matching	N	Y
Build and packaging	N	Y
Macros	N	Y
Closures	N	Y

Source: <https://pratapr.github.io/rust-over-c.html>

AUTHOR: Jaylem Brar



Why Rust?

Uses

- Commonly used to build operating systems
- Also low level programs that need to interact directly with operating systems
- Anything requiring high performance, think airplane code, robotics drivers, etc.
- Increasingly used to develop backend systems for web development though less common
- One of the most desired languages among developers and employers alike with rust blockchain devs being in high demand

Source: <https://github.blog/2023-08-30-why-rust-is-the-most-admired-language-among-developers/>

AUTHOR: Jaylem Brar



Why Rust?

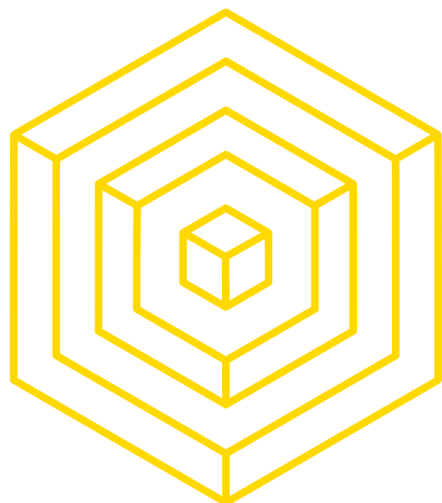
For blockchain

10

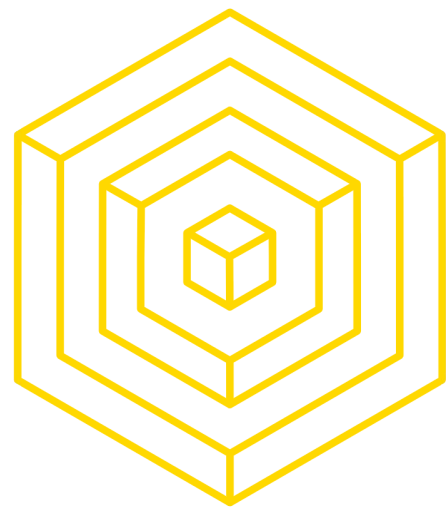
- Becoming a very popular language for blockchain development
- Valued here for it's good performance, strengths in concurrency, and strict memory management that prevents many memory related bugs
- Used in Polkadot, Near, and Solana among other projects
- Most popular development language on these chains

Source: <https://github.blog/2023-08-30-why-rust-is-the-most-admired-language-among-developers/>

AUTHOR: Jaylem Brar



2 Memory Management



Memory Management

Memory Addresses

- Memory contains 8 bit storage cells each with a unique address
- For 32 bit architecture has 4 byte words and each address is 4 bytes long
- i.e. 0xFFFFFFFF
- Commonly we need to access memory in 4 byte intervals
- Some processors won't allow otherwise or performance is hurt
- Structs in C are telling the machine how to arrange bytes in a bucket

Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



Memory Management

Memory Addresses

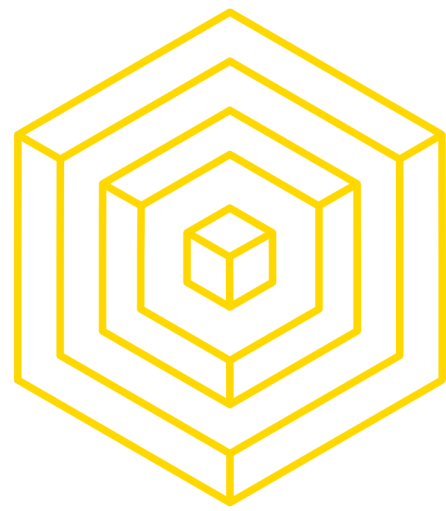
```
struct foo {  
    int32_t a;  
    char b;  
    struct foo *c;  
}
```



	4 bytes for c			
unused	unused	unused	1 byte for b	
	4 bytes for a			

Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



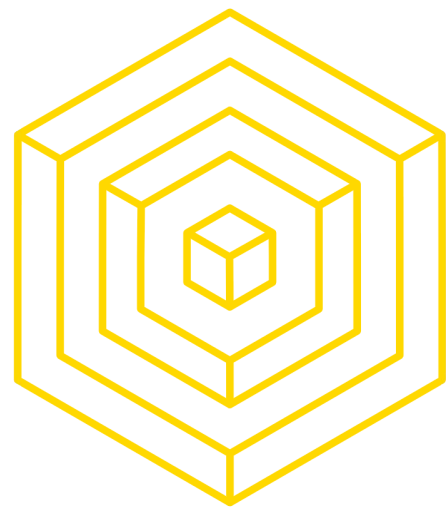
Memory Management

Regions of Memory

- Stack - this where local variables in functions are stored, grows downwards
- Heap - used for allocated memory, things like arrays and space here is allocated
- Static - Used for variables declared outside of main to store static variables
- Code - Used to store the program code

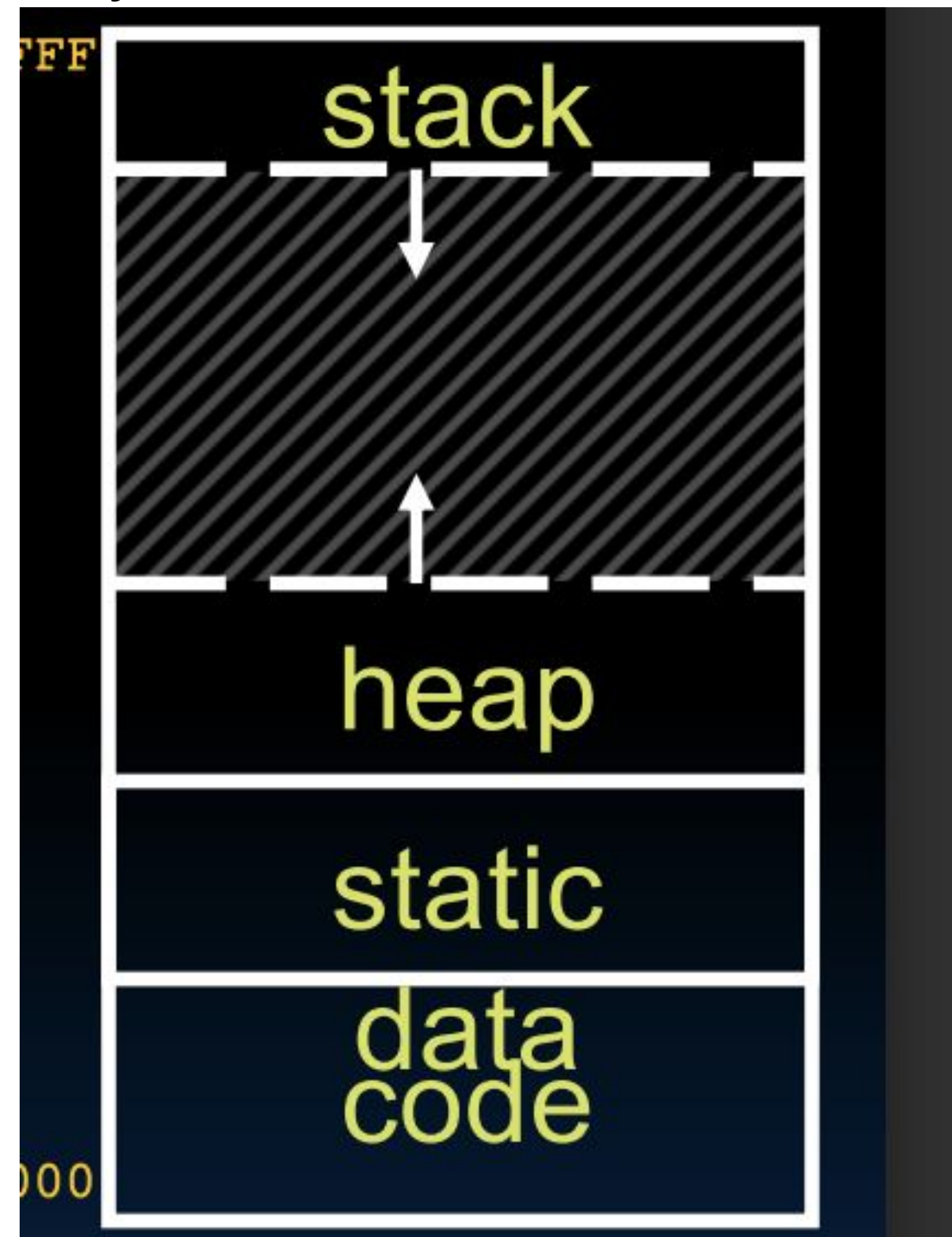
Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



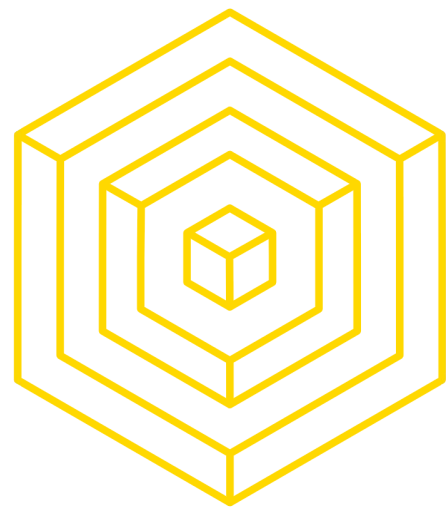
Memory Management

Regions of Memory



Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



Memory Management

The Stack

- A new stack frame is created with every new function call
- Contains information about a return instruction (where the function call was made)
- Also contains space for arguments to be passed
- And a stack pointer indicating the start of the stack frame
- In C these pointers can be edited for exploits, much more difficult with rust

Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



Memory Management

The Stack – Example

```
1  def g(a):  
2      x = a ** 2  
3      return x  
4  
5  def a(a, b):  
6      q = a + b + g(a + b)  
7      return q  
8  
9  print(a(5, 9))  
10  
11
```

Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



Memory Management

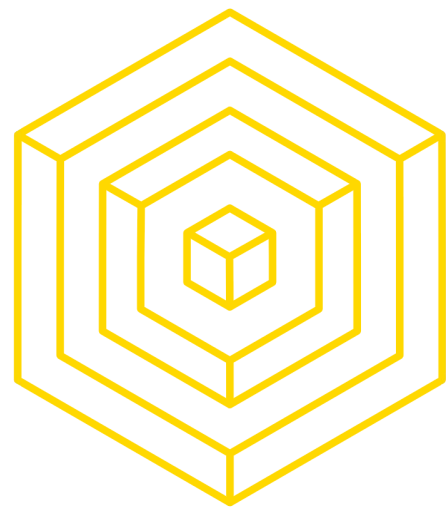
The Stack – Example

```
1  def g(a):  
2      x = a ** 2  
3      return x  
4  
5  def a(a, b):  
6      q = a + b + g(a + b)  
7      return q  
8  
9  print(a(5, 9))  
10  
11
```

- 5 and 9 are stored on the stack as arguments to a
- New stack frame created when a is called and again when g called
- x becomes local variable on the stack
- g returns execution to a
- q is evaluated and becomes local variable on the stack
- a returns
- Stack moves back up as returns happen and we lose those local variables

Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



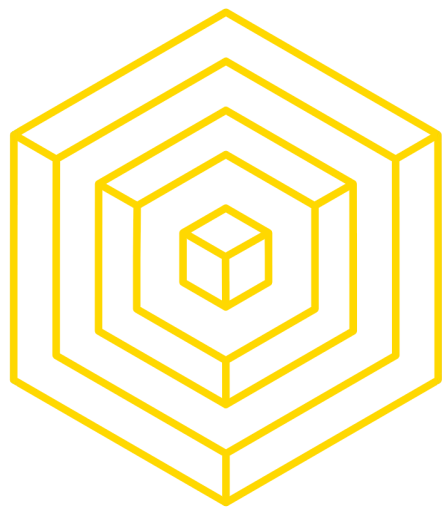
Memory Management

The Heap

- Information stored on the heap includes things such as arrays or other uninitialized memory
- Needs to be allocated and deallocated
- Done manually in C
- `X = malloc(256)`
- `free(X)`

Source: <https://cs61c.org/fa23/lectures/lec05/>

AUTHOR: Jaylem Brar



3 Memory Management in Rust



Memory Management in Rust

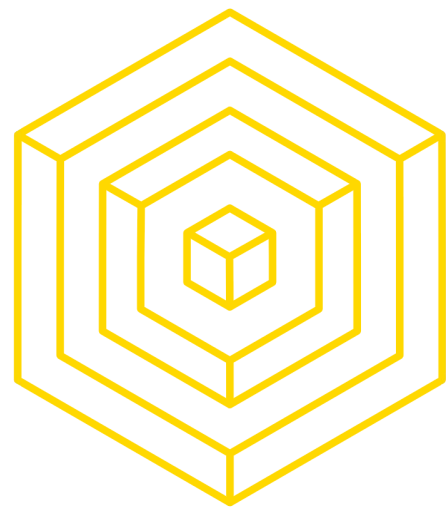
Improvements

- Ownership
- Different set of rules that does not require garbage collection or manually alloc and dealloc
- garbage collection is automatic checks for no-longer-used memory as the program runs, i.e. python java
- C is manual

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership

- Rules enforced with compiler checks
- If any of the rules are violated, the program won't compile
- Ownership does not slow down runtime and prevent memory errors before compilation

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

Key Concepts

- Each value in Rust has an owner
- There can only be one owner at a time
- When the owner goes out of scope, the value will be dropped

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership

- S is valid as long as it is in scope as seen in other languages

```
{           // s is not valid here, it's not yet declared
    let s = "hello"; // s is valid from this point forward

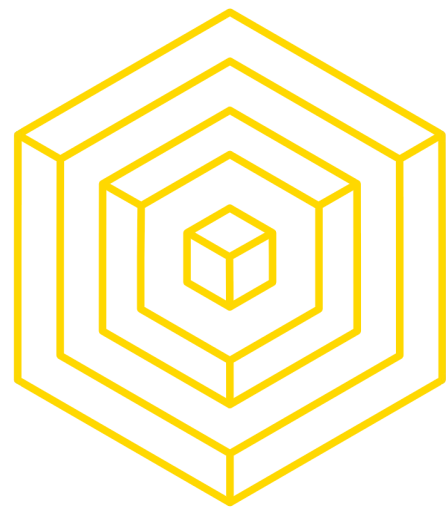
    // do stuff with s
}           // this scope is now over, and s is no longer valid
```



Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

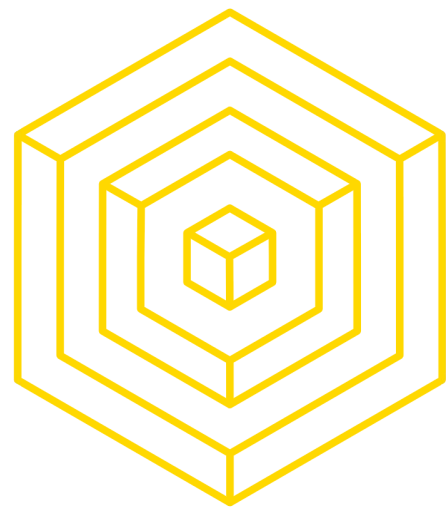
Ownership

- **let mut s = String::from("hello");**
- String type above is stored on heap as it has variable length
- The memory must be requested from the memory allocator at runtime.
- We need a way of returning this memory to the allocator when we're done with our String

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

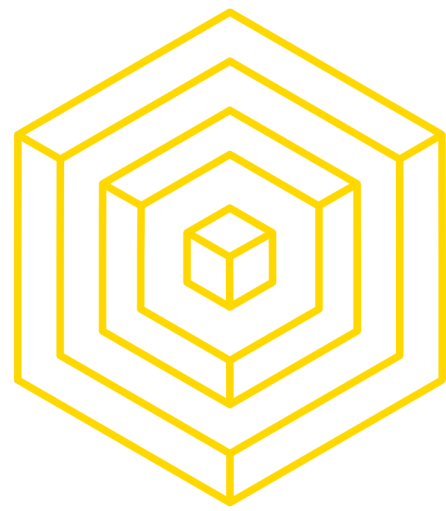
Ownership

- Allocation happens when we call the `String::from("hello");` like usual
- Deallocation happens automatically when the variable goes out of scope
- Rust automatically calls a special drop function for us

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership Trickiness

- Below $x = 5$ and then y is equal to x
- These fixed sized variables are stored on the stack

```
let x = 5;  
let y = x;
```

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership Trickiness

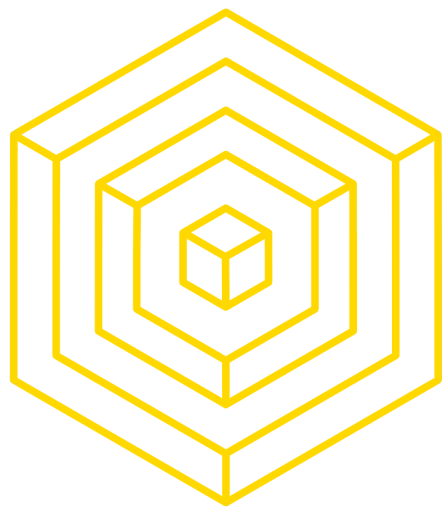
- This situation appears the same but is different
- These values are stored on the heap
- The heap region storing the string is not copied, only references

```
let s1 = String::from("hello");  
let s2 = s1;
```

Source:

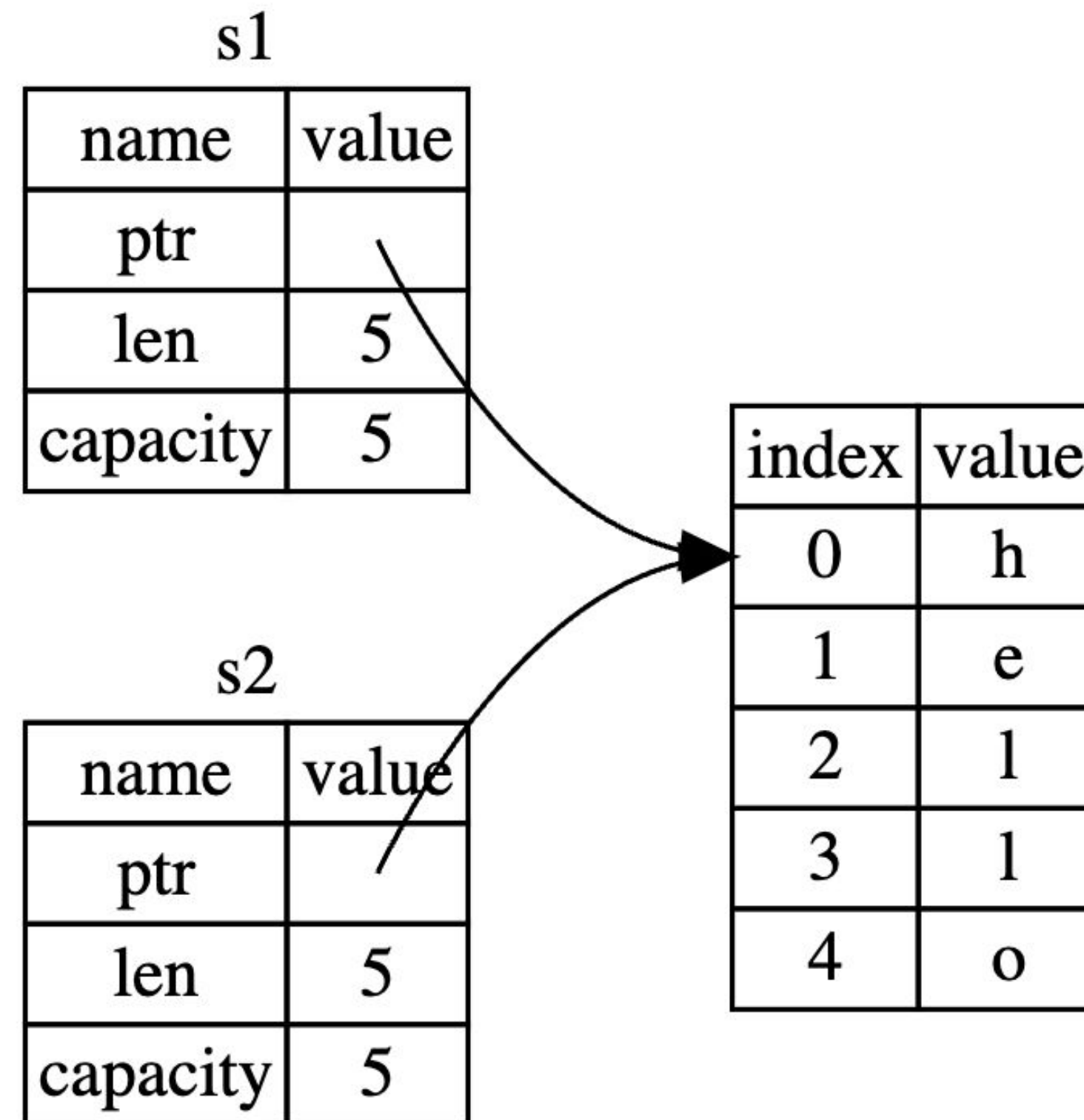
<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

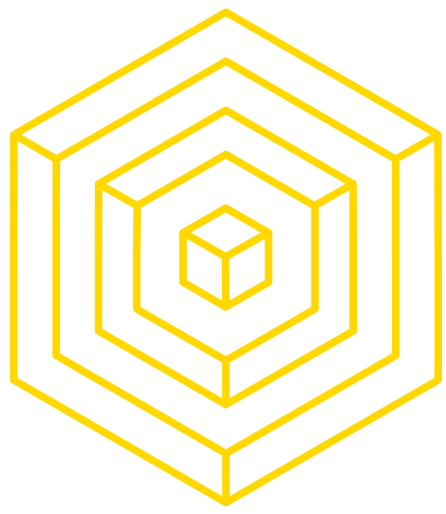
Ownership Trickiness



Source:

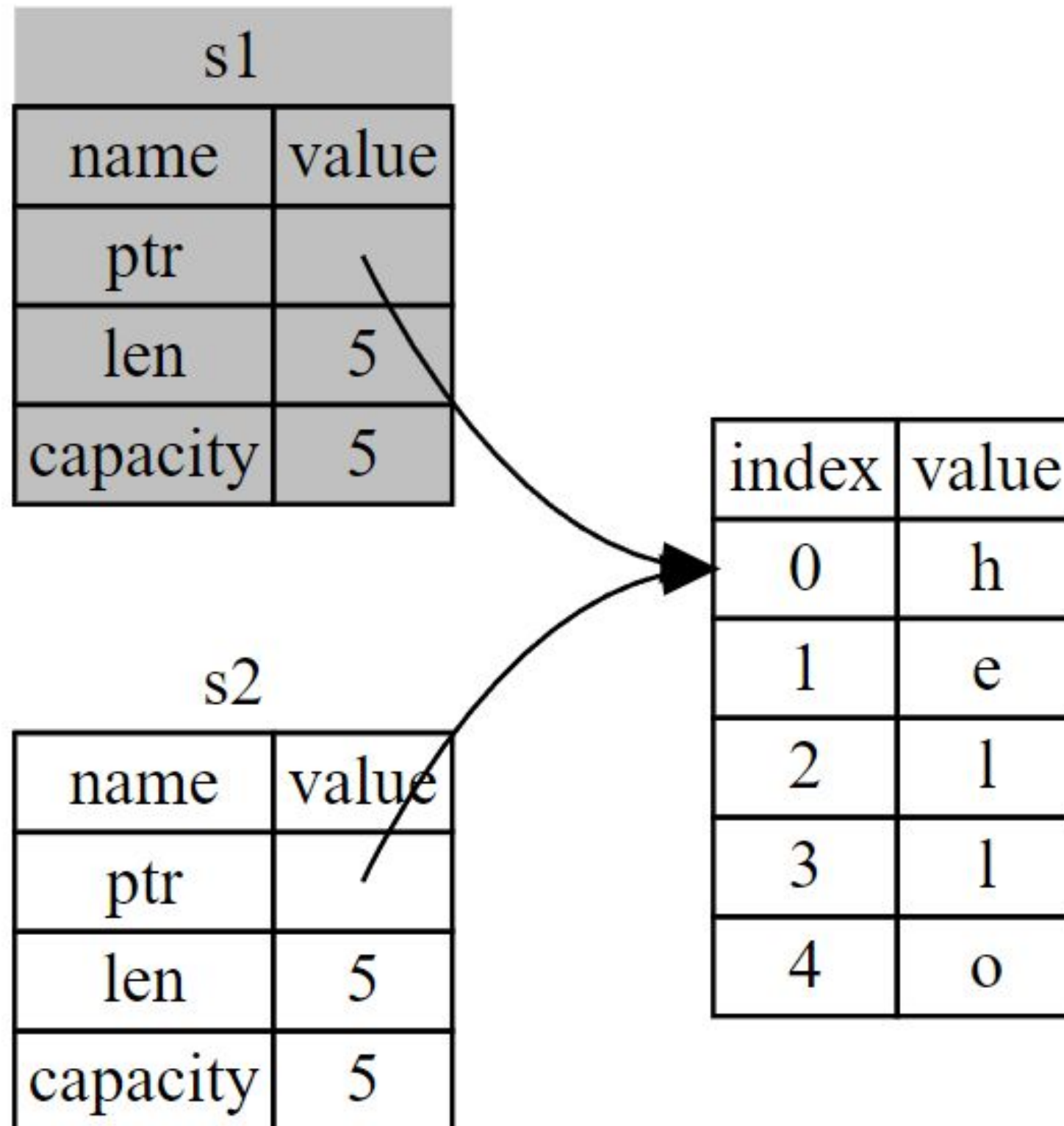
<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership Trickiness



Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

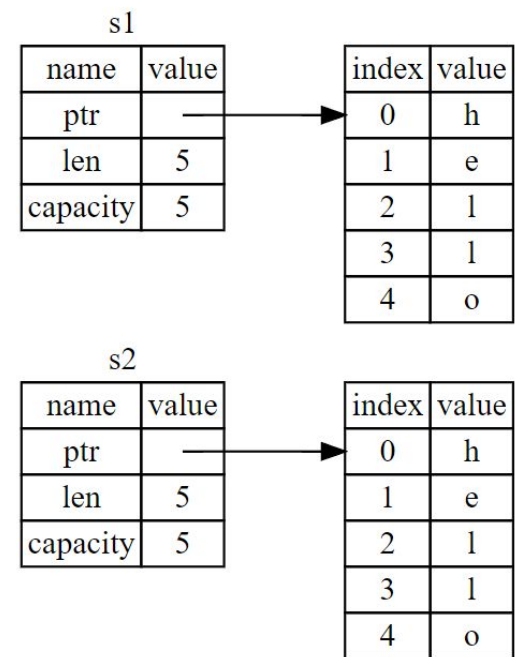
AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership Trickiness

- If we want deep copy behavior we can call clone



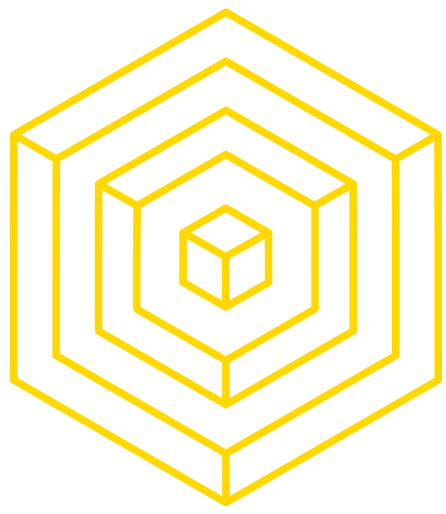
```
let s1 = String::from("hello");
let s2 = s1.clone();

println!("s1 = {}, s2 = {}", s1, s2);
```

Source:

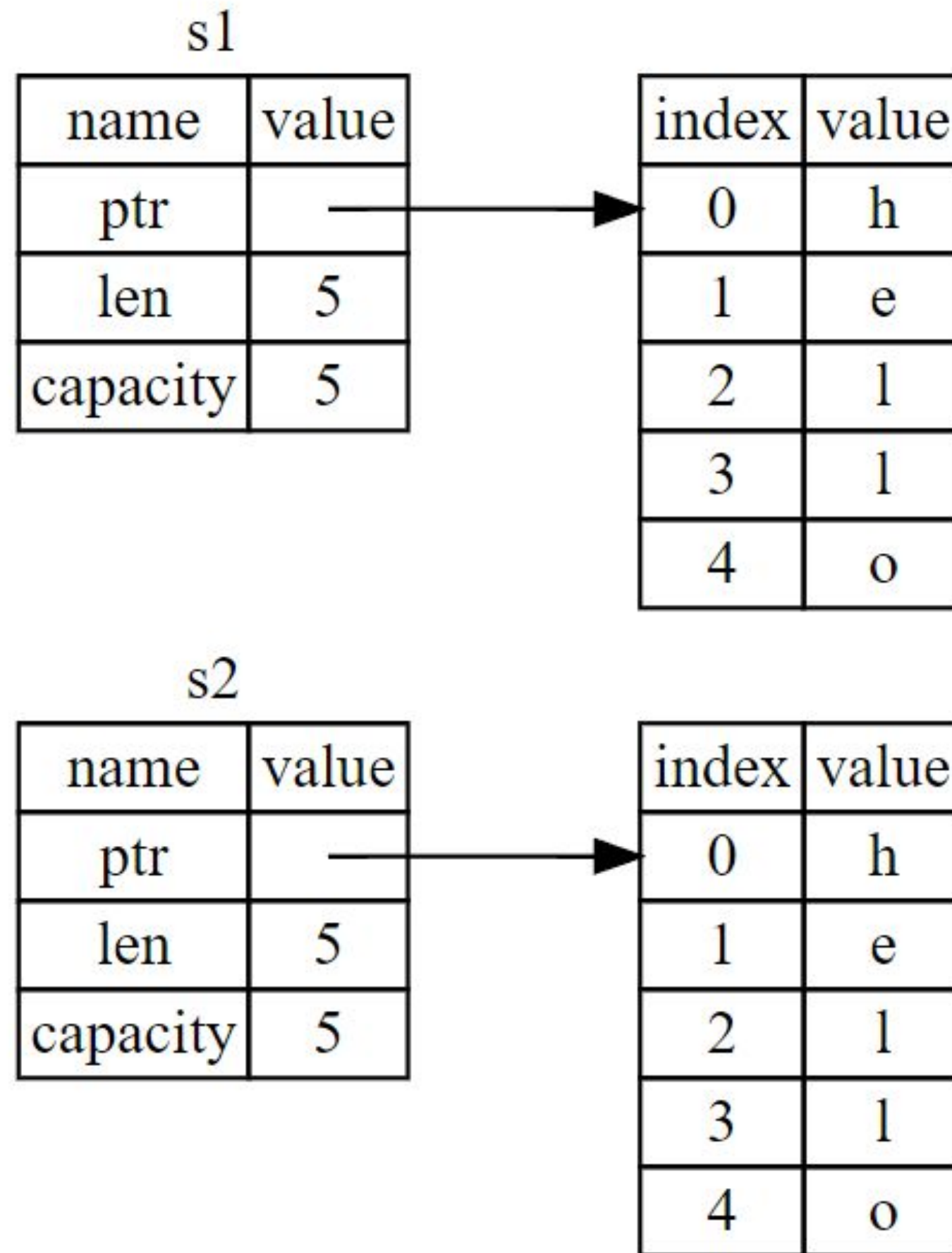
<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

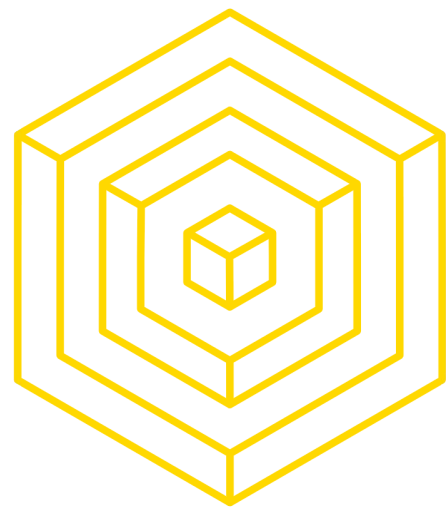
AUTHOR: Jaylem Brar



Memory Management in Rust

Ownership Trickiness





Memory Management in Rust

Ownership Trickiness

- A trait that can be placed on certain types
- Applies to Stack-only data, makes a physical copy
- Can't annotate types with Copy if they implement Drop
- Common types: integers, booleans, floats, chars

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar


```
fn main() {  
    let s = String::from("hello"); // s comes into scope  
  
    takes_ownership(s);             // s's value moves into the function...  
                                    // ... and so is no longer valid here  
  
    let x = 5;                       // x comes into scope  
  
    makes_copy(x);                  // x would move into the function,  
                                    // but i32 is Copy, so it's okay to still  
                                    // use x afterward  
  
} // Here, x goes out of scope, then s. But because s's value was moved, nothing  
  // special happens.  
  
fn takes_ownership(some_string: String) { // some_string comes into scope  
    println!("{}", some_string);  
} // Here, some_string goes out of scope and `drop` is called. The backing  
  // memory is freed.  
  
fn makes_copy(some_integer: i32) { // some_integer comes into scope  
    println!("{}", some_integer);  
} // Here, some_integer goes out of scope. Nothing special happens.
```



```

fn main() {
    let s1 = gives_ownership();           // gives_ownership moves its return
                                          // value into s1

    let s2 = String::from("hello");      // s2 comes into scope

    let s3 = takes_and_gives_back(s2);   // s2 is moved into
                                          // takes_and_gives_back, which also
                                          // moves its return value into s3
} // Here, s3 goes out of scope and is dropped. s2 was moved, so nothing
  // happens. s1 goes out of scope and is dropped.

fn gives_ownership() -> String {         // gives_ownership will move its
                                          // return value into the function
                                          // that calls it

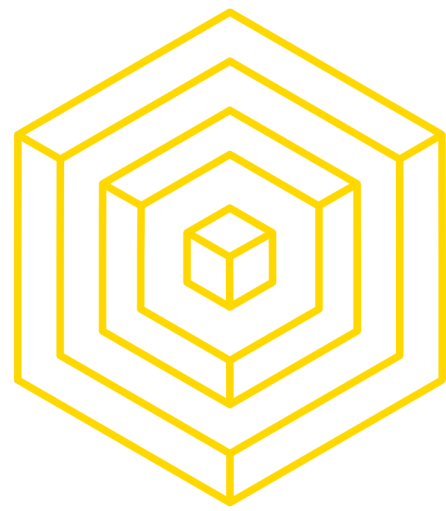
    let some_string = String::from("yours"); // some_string comes into scope

    some_string                           // some_string is returned and
                                          // moves out to the calling
                                          // function
}

// This function takes a String and returns one
fn takes_and_gives_back(a_string: String) -> String { // a_string comes into
                                                         // scope

    a_string // a_string is returned and moves out to the calling function
}

```

Memory Management in Rust

Ownership Trickiness

- How to avoid this constant transfer of ownership?
- We'll see next time

Source:

<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html#:~:text=Rust%20uses%20a%20third%20approach,your%20program%20while%20it's%20running.>

AUTHOR: Jaylem Brar



4

Other Rust Syntax



Syntax in Rust

Returning Values

- In Rust, you don't need to include a “return” statement
 - The final expression is automatically returned if you omit a semicolon
 - “Return” usually only used to exit a function early

```
fn square(x: i32) -> i32 {  
    x * x  
}
```

Source: <https://doc.rust-lang.org/reference/expressions/loop-expr.html>

AUTHOR: Brent Friedman



Syntax in Rust

Rust is expression-oriented

- Most constructs return a value
 - Different from other languages which have statements (do something) and expressions (return a value)

```
let x = if some_condition {  
    5  
} else {  
    10  
};
```





Syntax in Rust

Match Expression

- Can think of this as a switch-case
 - `_` is the default action

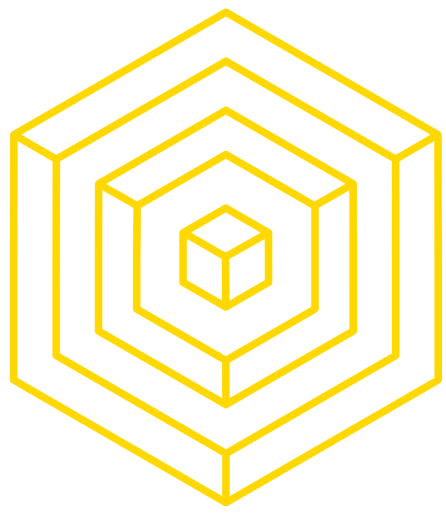
An example of a `match` expression:

```
let x = 1;

match x {
    1 => println!("one"),
    2 => println!("two"),
    3 => println!("three"),
    4 => println!("four"),
    5 => println!("five"),
    _ => println!("something else"),
}
```

Source: <https://doc.rust-lang.org/reference/expressions/loop-expr.html>

AUTHOR: Brent Friedman



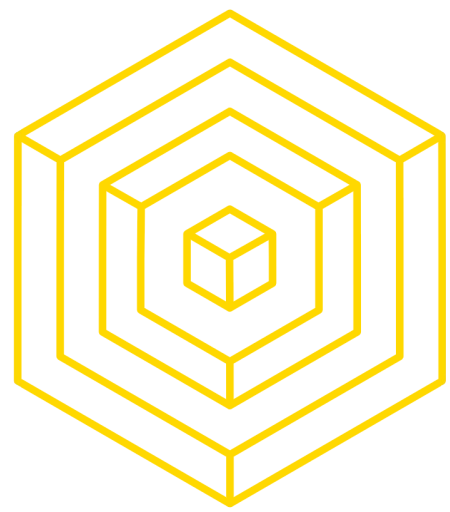
Syntax in Rust

Rust is expression-oriented

- Result enum type in practice

```
fn divide(a: f64, b: f64) -> Result<f64,
    &'static str> {
    if b == 0.0 {
        return Err("Division by zero");
    }
    Ok(a / b)
}

let result = divide(10.0, 2.0);
match result {
    Ok(value) => println!("Result is: {}",
value),
    Err(error) => println!("Error: {}",
error),
}
```

Syntax in Rust

Panic!

- Panic!
 - Ends the program
 - Used to prevent memory leaks and other errors

```
panic!();  
panic!("this is a terrible mistake!");  
panic!("this is a {} {message}", "fancy", message = "message");
```

 <https://doc.rust-lang.org/std/macro.panic.html>

AUTHOR: Brent Friedman